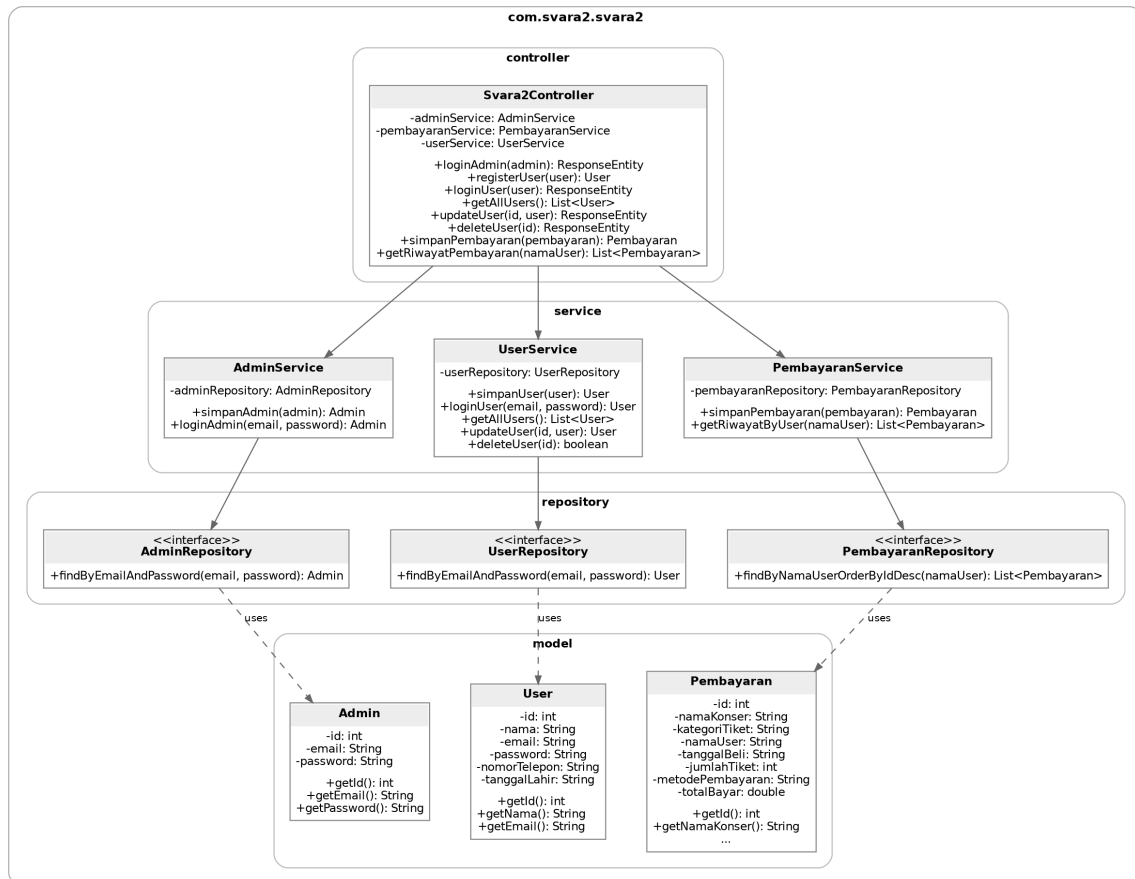


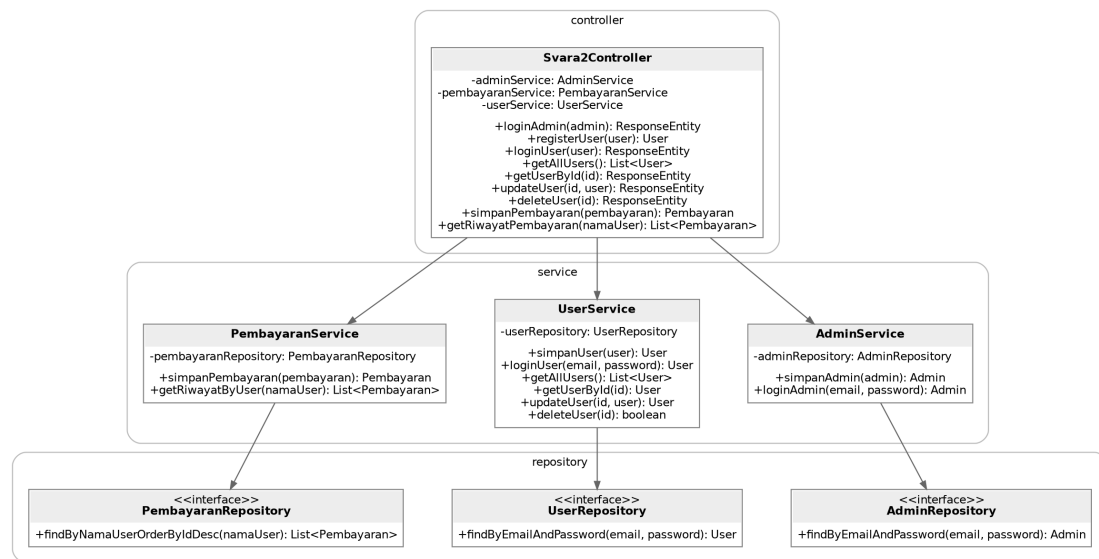
Class Diagram

1. Skema Class Diagram

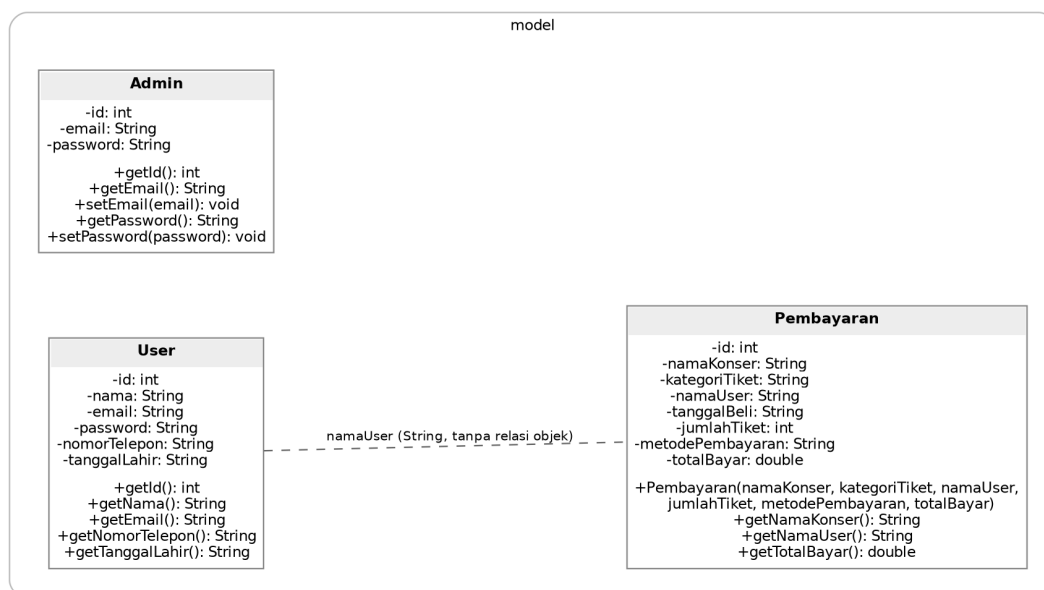
Diagram kelas berikut menggambarkan rancangan sistem *Svara2*, sebuah aplikasi untuk pemesanan sekaligus pembayaran tiket konser, mencakup lapisan *backend* pada sisi backend (Model, Repository, Service, dan Controller).



Gambar berikut memperlihatkan relasi antara Controller, Service, dan Repository:



Gambar berikut memperlihatkan relasi antar kelas di dalam Model:



2. Penjelasan Lengkap Class Diagram Proyek

Struktur kode di atas menerapkan pola *Layered Architecture* (arsitektur berlapis) yang umum dipakai pada ekosistem Spring Boot. Kelas-kelas dikelompokkan ke dalam paket (*packages*) agar tiap lapisan fokus pada tanggung jawabnya sendiri, sesuai prinsip *Separation of Concerns* (SoC), sehingga kode tetap modular dan mudah dirawat.

Penjelasan tiap komponen dijabarkan sebagai berikut:

A. Lapisan Model

Lapisan ini menampung objek data bisnis (*Plain Old Java Object* / POJO) yang dipetakan ke tabel database lewat anotasi JPA/Hibernate (`@Entity`, `@Table`, `@Id`).

1. Admin: menampung kredensial administrator (*email* dan *password*) yang dipakai khusus untuk login admin, terpisah dari tabel pengguna biasa.
2. User: menyimpan data akun pengguna aplikasi, termasuk data pribadi seperti *nama*, *nomorTelepon*, dan *tanggalLahir*, yang dipakai saat registrasi maupun login.
3. Pembayaran: mencatat setiap transaksi pembelian tiket konser. Berlainan dengan pola relasi berbasis objek (contohnya relasi *Booking* → *User* pada sistem penyewaan), kelas ini menyimpan *namaUser* sebagai tipe `String` secara langsung sebagai teks (denormalisasi), bukan sebagai referensi ke objek *User*. Cara ini membuat tiap catatan transaksi berdiri sendiri, tanpa bergantung pada relasi *foreign key*.

B. Lapisan Repositori

Seluruh kelas pada paket ini berbentuk antarmuka (*interface*) yang meng-extend `JpaRepository` dari Spring Data JPA, sehingga operasi CRUD dasar tidak perlu ditulis manual dalam SQL.

1. `AdminRepository` dan `UserRepository` masing-masing memiliki metode turunan `findByEmailAndPassword()` yang menjadi dasar proses login berbasis email dan password.
2. `PembayaranRepository` menyediakan metode `findByNamaUserOrderByIdDesc()` untuk menampilkan riwayat pembelian tiket seorang pengguna, diurutkan dari yang paling baru.

C. Lapisan Service

Paket ini berisi kelas-kelas konkret yang menjalankan logika bisnis utama, masing-masing memanggil (*inject*) repositori yang sesuai melalui `@Autowired`.

1. `AdminService`: mengurus penyimpanan data admin dan pengecekan login admin.
2. `UserService`: menangani seluruh siklus akun pengguna, mulai dari registrasi, login, pengambilan data (semua maupun satu pengguna), pembaruan data, sampai penghapusan akun.
3. `PembayaranService`: mengatur alur pencatatan pembayaran. Sebelum data disimpan, kelas ini otomatis mencatat waktu transaksi memakai `SimpleDateFormat` dengan locale Indonesia, lalu menyerahkan penyimpanannya ke `PembayaranRepository`.

D. Lapisan Controller (API View Layer)

Kelas `Svara2Controller` menjadi satu-satunya pintu masuk komunikasi dari luar sistem (`@RestController`). Kelas ini menangani *HTTP Request* yang datang dari klien, mengubah JSON menjadi objek Java lewat `@RequestBody`, lalu mengembalikan responsnya dibungkus `ResponseEntity` lengkap dengan kode status HTTP (200 OK, 401 Unauthorized, 404 Not Found, dst). Lapisan ini hanya meneruskan alur data, tanpa logika bisnis di dalamnya.

3. Pola Relasi Antar Kelas dalam Sistem

Diagram ini menunjukkan dua jenis relasi utama:

1. Relasi longgar pada lapisan Model (User – Pembayaran): tidak seperti sistem yang menghubungkan objek lewat referensi langsung, **Pembayaran** di sini cukup menyimpan **namaUser** sebagai teks biasa. Desain ini lebih ringkas dan cukup untuk mencatat riwayat transaksi tanpa perlu *join* relasional yang rumit.
2. Alur Ketergantungan Antar Komponen (Dependency Injection lewat tanda panah →): mencerminkan prinsip *Loose Coupling*. Kelas pada lapisan atas selalu bergantung pada kelas di lapisan tepat di bawahnya:
 - Controller memanggil ketiga kelas Service (AdminService, UserService, PembayaranService).
 - Tiap Service memanggil satu Repository yang sesuai dengannya.
3. Pola delegasi satu arah ini membuat proses penelusuran kesalahan (*debugging*) lebih mudah ditebak, memudahkan pengujian unit, dan menjaga kode tetap rapi saat aplikasi berkembang lebih besar.